Лабораторна робота № 1

Тема: «Дослідження реалізацій протоколу SSL/TLS».

Обрана вразливість: Heartbleed.

Виконали: Остаповець Олеся Віталіївна та Булигін Олексій Михайлович,
група ФІ-52мн.

Мета роботи та використані засоби

Мета роботи — дослідити особливості реалізації криптографічних механізмів TLS, встановлення захищеного з'єднання, використання цифрових сертифікатів та протидії атакам на захищений канал зв'язку.

Досліджено параметри HTTPS-з'єднання з GitHub у браузері та результати онлайн-перевірки Cloudflare. Проаналізовано сучасні механізми TLS, вразливість Heartbleed і наслідки вимкнення перевірки сертифіката сервера.

Використані засоби: браузер на основі Chromium у Windows, панель DevTools Security і вбудований переглядач сертифікатів; онлайн-сервіси digga та teardown.tools для перевірки другого ресурсу. Результати зафіксовано 07.10.2026.

1. Дослідження TLS-з'єднання

1.1. GitHub

Досліджуваний ресурс: github.com. Параметри з'єднання та сертифіката наведено в таблиці 1 й підтверджено знімками екрана (рис. 1–6). Панель Security показує TLS 1.3, X25519 та AES-128-GCM. Повну назву cipher suite й назву кривої ключа додатково звірено з TLS-оглядом digga [11].

Таблиця 1 — Параметри з'єднання та сертифіката github.com

Параметр	Значення
Версія TLS	TLS 1.3
Cipher suite	TLS_AES_128_GCM_SHA256 [11]
Суб'єкт сертифіката	github.com
Центр сертифікації	Sectigo Public Server Authentication CA DV E36 — Sectigo Limited
Строк дії	01.09.2026 03:00:00– 30.11.2026 01:59:59 (час Києва, як у браузері)
SAN	github.com; www.github.com
Відкритий ключ	EC; prime256v1 (P-256), 256 біт [11]
Підпис сертифіката	ECDSA із SHA-256 (ecdsa-with-SHA256)

Ланцюжок сертифікації

github.com → Sectigo Public Server Authentication CA DV E36 → Sectigo Public Server Authentication Root E46.

У переглядачі сертифікатів браузер побудував шлях до довіреного кореня Sectigo Public Server Authentication Root E46 (рис. 3). Підписи проміжного й серверного сертифікатів пов’язують відкритий ключ сайту з цим коренем довіри.

Аналіз параметрів

TLS_AES_128_GCM_SHA256 визначає AES із 128-бітним ключем у режимі GCM для шифрування та перевірки цілісності записів. SHA-256 використовується в HKDF і для хешування повідомлень узгодження. Обмін ключами в спостережуваному з’єднанні виконується через X25519. Ключ сертифіката належить до EC, а підпис сертифіката — ECDSA із SHA-256 (рис. 4–6); ці алгоритми узгоджуються окремо від cipher suite [1–2].

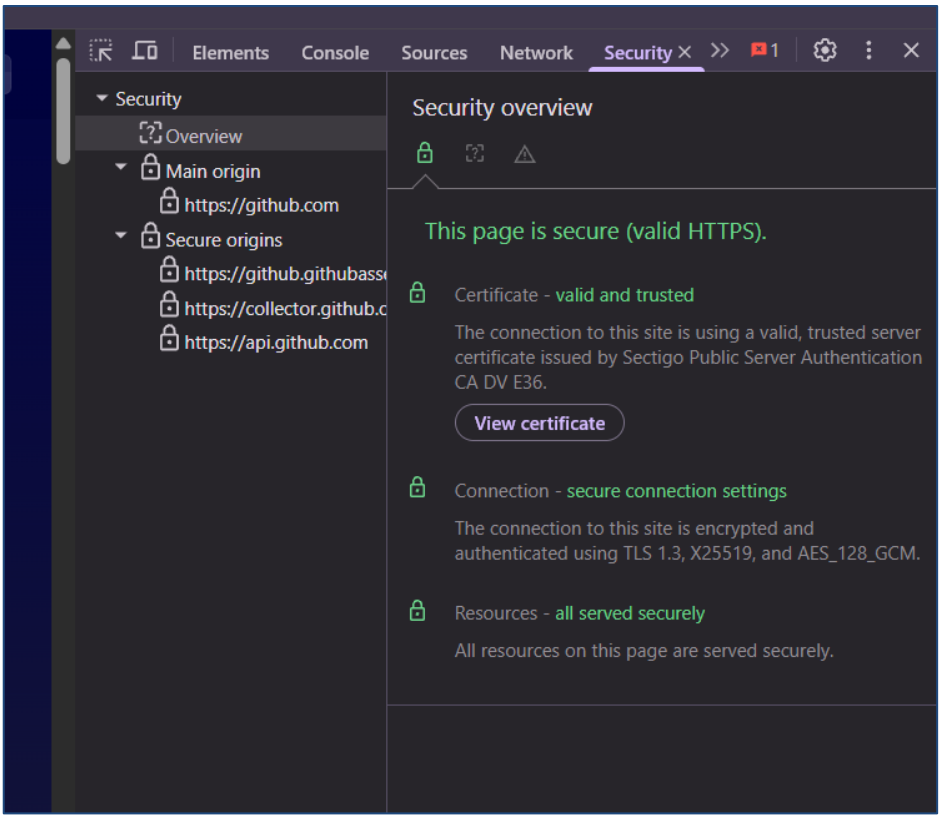


Рисунок 1 — Параметри захищеного з’єднання з github.com

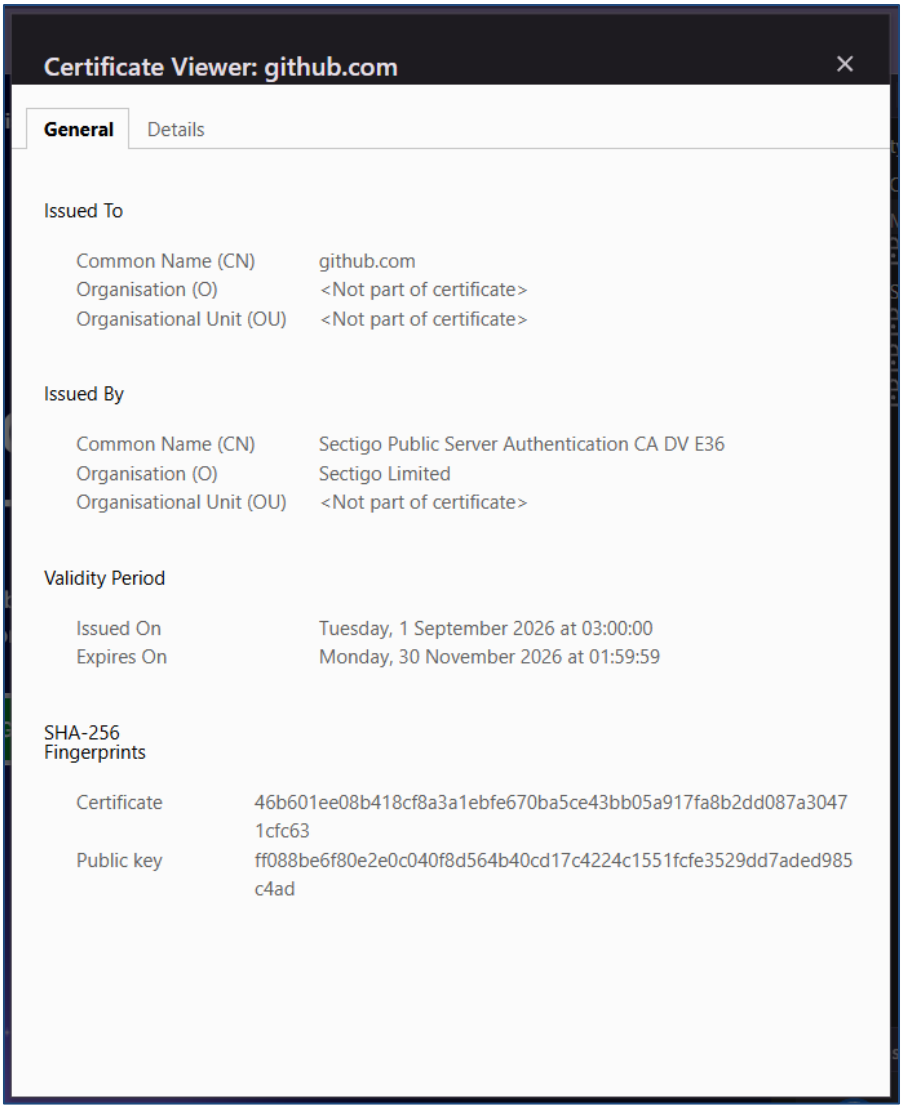


Рисунок 2 — Суб’єкт, видавець і строк дії сертифіката github.com

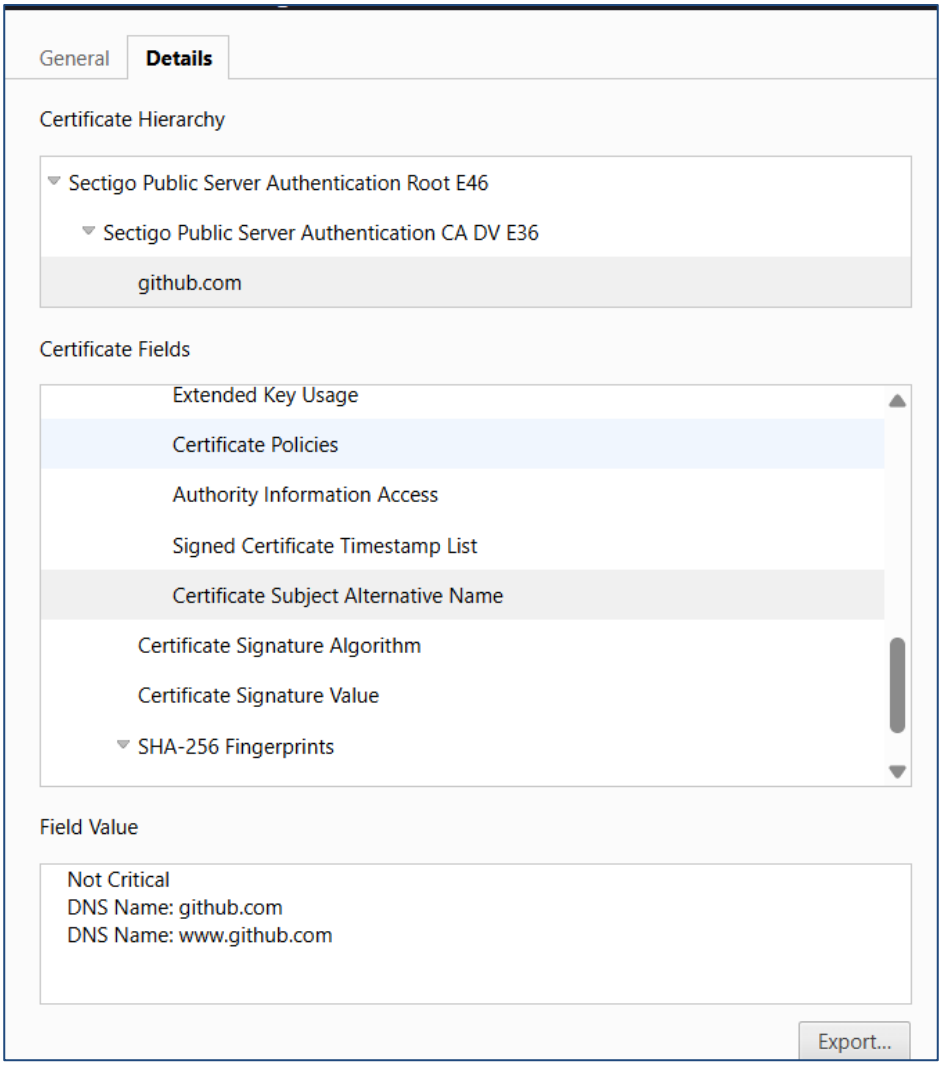


Рисунок 3 — Ланцюжок сертифікації та поле SAN сертифіката github.com

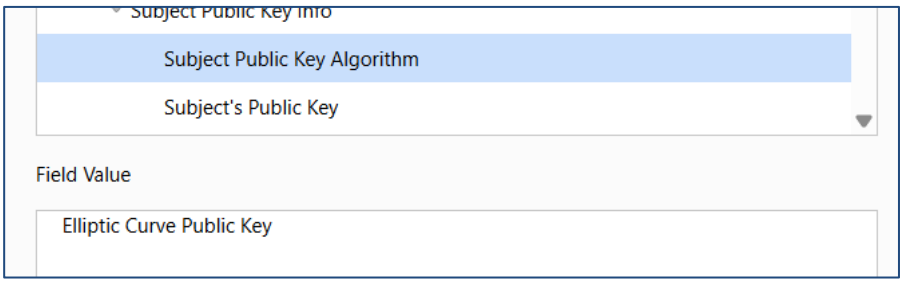


Рисунок 4 — Алгоритм відкритого ключа сертифіката github.com

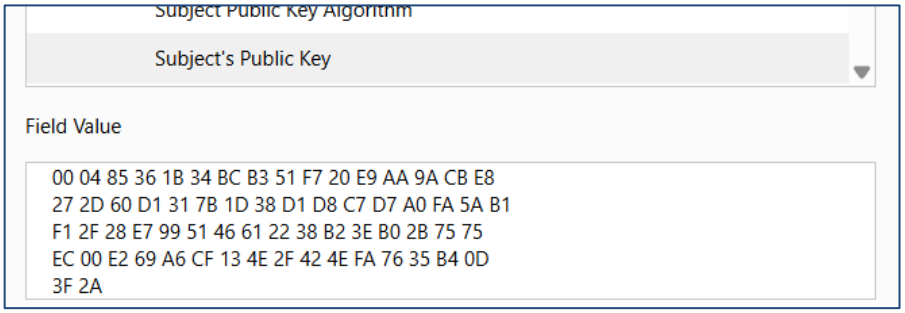


Рисунок 5 — Значення відкритого ключа сертифіката github.com

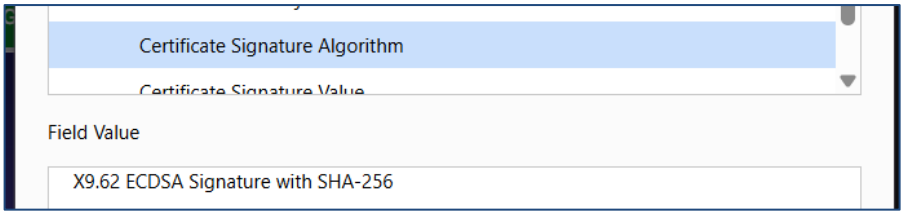


Рисунок 6 — Алгоритм цифрового підпису сертифіката github.com

1.2. Cloudflare

Досліджуваний ресурс: cloudflare.com. Параметри отримано з онлайн-оглядів digga та teardown.tools [12, 14] і зведено в таблиці 2. В обох оглядах наведено сертифікат із серійним номером D509925C8DB10E1913A3A1CB1C7B05AB; значення cipher suite характеризує з'єднання перевірочного сервісу.

Таблиця 2 — Параметри з'єднання та сертифіката cloudflare.com

Параметр	Значення
Версія TLS	TLS 1.3
Cipher suite	TLS_AES_256_GCM_SHA384
Суб'єкт сертифіката	cloudflare.com
Центр сертифікації	WE1 — Google Trust Services
Початок дії	05.09.2026 22:29:39 UTC [14]
Закінчення дії	04.12.2026 23:29:33 UTC [14]
SAN	cloudflare.com; ns.cloudflare.com; *.ns.cloudflare.com; *.secondary.cloudflare.com; secondary.cloudflare.com
Відкритий ключ	EC; prime256v1 (P-256), 256 біт
Підпис сертифіката	ECDSA із SHA-256 (ecdsa-with-SHA256) [14]

Ланцюжок сертифікації

cloudflare.com → WE1 → GTS Root R4 → GlobalSign Root CA.

В онлайн-огляді наведено шлях через перехресно підписаний GTS Root R4 до GlobalSign Root CA [12, 14]. Клієнт, який безпосередньо довіряє GTS Root R4, може завершити перевірку на цьому корені.

Аналіз параметрів

TLS_AES_256_GCM_SHA384 задає AES-256-GCM та SHA-384. Автентичність сертифіката перевіряється окремо від захисту TLS-записів: цифровий підпис центру сертифікації не є тегом GCM. Значення SHA-384 у cipher suite також не означає, що сертифікат підписано із SHA-384 [1].

Обидва досліджені з’єднання використовують TLS 1.3 та AEAD-режим GCM. Вони мають різні cipher suites і центри сертифікації. Узгоджений набір характеризує конкретне з’єднання, а не весь перелік алгоритмів, які підтримує сервер.

Зведений фрагмент результатів онлайн-перевірки cloudflare.com [12, 14]:

```
Протокол: TLSv1.3
Cipher suite: TLS_AES_256_GCM_SHA384
Видавець: WE1 (Google Trust Services)
Відкритий ключ: EC, P-256, 256 біт
Підпис: ECDSA із SHA-256
```

1.3. Призначення параметрів сертифіката

Subject позначає суб’єкта сертифіката; Issuer — центр, який його підписав. Not Before і Not After обмежують строк дії. SAN містить дозволені доменні імена та використовується для перевірки відповідності адреси сервера. Відкритий ключ дозволяє перевірити володіння відповідним приватним ключем, а підпис CA — цілісність сертифіката. Ланцюжок довіри пов’язує сертифікат сервера з довіреним коренем [4].

1.4. Встановлення TLS-з’єднання

У з’єднанні GitHub узгоджено TLS 1.3 та X25519 (рис. 1). Для повного встановлення TLS 1.3-з’єднання з обміном ECDHE й автентифікацією сервера сертифікатом основні етапи такі [1]:

1. ClientHello: клієнт пропонує версії, cipher suites, алгоритми підпису та групи обміну ключами; передає key_share.

2. ServerHello: сервер обирає параметри й надсилає свою частку обміну ключами. За потреби HelloRetryRequest запитує іншу частку.

3. Формування ключів: сторони обчислюють спільний ECDHE-секрет і через HKDF отримують окремі ключі для напрямків передавання [2]. Приватні ефемерні ключі мережею не передаються.

4. Сервер надсилає EncryptedExtensions, Certificate, CertificateVerify та Finished. Клієнт перевіряє ім’я, строк дії, ланцюжок і підписи; CertificateVerify підтверджує володіння приватним ключем.

5. Клієнт перевіряє Finished і надсилає власне Finished. Ці повідомлення підтверджують узгодження ключового матеріалу та цілісність повідомлень handshake; прикладні дані захищаються окремими ключами трафіку.

Після ServerHello основні повідомлення handshake шифруються. Ключ сертифіката та ефемерний ключ обміну виконують різні функції; їх не слід ототожнювати.

2. Аналіз сучасних механізмів TLS

2.1. Відмінності TLS 1.3 від TLS 1.2

Таблиця 3 — Порівняння TLS 1.2 і TLS 1.3

Механізм	TLS 1.2	TLS 1.3
Захист записів	AEAD або старіші режими, зокрема CBC	Лише AEAD
Обмін ключами	Можливий статичний RSA	Статичний RSA видалено
Назва cipher suite	Містить більше складових	AEAD та хеш
Handshake	Типово 2 RTT	Типово 1 RTT
Похідні ключі	PRF	HKDF
Відновлення сеансу	Без стандартного 0-RTT	Можливий 0-RTT із ризиком повторного відтворення

Порівняння базується на [1–3]. TLS 1.2 із коректними сучасними налаштуваннями може забезпечувати захищений канал.

2.2. Чому SSL та ранні TLS застаріли

SSL 2.0 і SSL 3.0 мають фундаментальні проблеми безпеки; SSL 3.0, зокрема, пов’язаний з атакою POODLE на CBC padding. RFC 7568 забороняє використання SSL 3.0 [6]. TLS 1.0 і TLS 1.1 офіційно виведено з використання RFC 8996: вони не відповідають сучасним вимогам до алгоритмів і механізмів захисту та збільшують площу атаки [5].

2.3. Forward Secrecy

Forward Secrecy означає, що подальший витік довгострокового ключа автентифікації не розкриває записані раніше сеанси за умови використання ефемерного обміну та знищення його секретів. У TLS 1.3 це забезпечується (EC)DHE; режим лише PSK без (EC)DHE не дає такої гарантії, а для 0-RTT вона також обмежена [1]. Витік самого сеансового ключа або доступ до пам’яті процесу цією властивістю не усувається.

2.4. Certificate Transparency

Certificate Transparency — публічні журнали TLS-сертифікатів. Криптографічні докази дозволяють перевіряти включення сертифіката і узгодженість журналу. Власники доменів можуть виявляти помилкову чи несанкціоновану видачу. CT підвищує підзвітність CA, але не замінює перевірку довіри чи відкликання сертифіката [8].

2.5. CRL та OCSP

CRL — підписаний CA список відкликаних сертифікатів. Клієнт перевіряє серійний номер у чинному списку; обмеження — розмір CRL, потреба завантаження та затримка оновлення [4]. OCSP надає підписаний статус окремого сертифіката: good, revoked або unknown. Good саме по собі не означає повної валідності сертифіката: інші перевірки залишаються необхідними [7].

OCSP залежить від доступності responder та свіжості відповіді; прямий запит також може розкривати інформацію про звернення користувача. OCSP stapling дозволяє серверу передати підписану відповідь клієнту. У разі мережевої помилки політика soft-fail може дозволити з'єднання без підтвердження статусу [7].

2.6. HSTS

Через заголовок Strict-Transport-Security сервер повідомляє браузеру, що домен потрібно відкривати лише через HTTPS. Це протидіє sslstrip і забороняє обхід помилки сертифіката для HSTS-домену. Без попередньо збереженої політики чи preload перший HTTP-візит залишається вразливим. HSTS не усуває помилки криптографічної бібліотеки [9].

2.7. Навіщо перевіряти сертифікат

Шифрування приховує зміст від стороннього спостерігача, але саме по собі не підтверджує особу сервера. Перевірка імені, ланцюжка та чинності сертифіката пов'язує отриманий ключ із потрібним ресурсом. Без неї активний посередник може створити два окремі TLS-сеанси та отримувати відкриті дані між ними [1, 4].

3. Дослідження вразливостей SSL/TLS

3.1. Сутність вразливості Heartbleed

Heartbleed — CVE-2014-0160, помилка реалізації розширення TLS/DTLS Heartbeat у OpenSSL. Heartbeat дозволяє перевірити доступність іншої сторони, надсилаючи дані, які та повинна повернути. У вразливій реалізації перевірка заявленої довжини payload була недостатньою, що спричиняло читання за межами буфера [10, 13].

Наприклад, фактично передано короткий payload, але поле довжини повідомляє про значно більший обсяг. Сервер копіює заявлену кількість байтів: разом із payload у відповідь потрапляє сусідня пам'ять процесу. Один запит міг розкрити до приблизно 64 КБ пам'яті; повторні запити могли розкривати інші фрагменти [10].

3.2. Умови реалізації

Необхідні вразлива бібліотека OpenSSL із підтримкою Heartbeat та можливість надіслати їй спеціально сформоване повідомлення через TLS/DTLS. Для витоку з сервера не потрібні облікові дані користувача або MITM. Вразливість також стосувалася клієнтів, що підключалися до зловмисного сервера [13].

Уразливі стабільні випуски — OpenSSL 1.0.1–1.0.1f. Виправлення вийшло в 1.0.1g 7 квітня 2014 року; проблема також зачіпала ранні beta-випуски 1.0.2 [10, 13].

3.3. Причина та наслідки

Причина — buffer over-read через недостатню перевірку меж, а не злам AES, RSA або математичного принципу TLS. З пам'яті могли витікати паролі, cookies, токени, дані користувачів і ключовий матеріал, зокрема приватні ключі. Витік конкретного секрету залежав від його наявності в прочитаній ділянці пам'яті [13].

Forward Secrecy не захищає секрети, які атакувальник безпосередньо читає з пам’яті активного процесу. Водночас витік лише довгострокового ключа не розкриває раніше записані сеанси з коректним ефемерним обміном.

3.4. Усунення та актуальність для TLS 1.3

Виправлення додає перевірку, що повідомлення справді містить заявлений payload і необхідний padding [10]. Після виявлення вразливої системи слід оновити бібліотеку, перезапустити залежні служби, замінити потенційно викрадені ключі, перевидати й відкликати сертифікати та скинути скомпрометовані токени й паролі [13].

OpenSSL 1.0.1g є історичною точкою виправлення; для сучасної системи потрібна підтримувана версія з актуальними оновленнями. Heartbleed не є вразливістю TLS 1.3: уражена гілка передувала його реалізації. Виправлення забезпечує коректна бібліотека, а сама зміна версії протоколу не замінює оновлення програмного забезпечення.

4. Аналіз ситуації порушення безпеки

Умова: клієнт отримує помилку перевірки сертифіката сервера. Розробник вимикає перевірку, після чого TLS-з’єднання встановлюється, а трафік передається зашифрованим.

4.1. Чому з’єднання стало можливим

Клієнт припинив вимагати підтвердження довіри до сертифіката або його відповідності серверу. Якщо початковою причиною був недовірений CA, неправильне ім’я, прострочення чи неповний ланцюжок, ця причина більше не зупиняє з’єднання. Криптографічний обмін може завершитися, хоча особу потрібного сервера не підтверджено.

4.2. Які властивості зберігаються

Шифрування та перевірка цілісності TLS-записів діють у встановленому сеансі, але автентифікація потрібного сервера втрачається. Пасивний спостерігач зазвичай не може прочитати трафік. Захист конфіденційності та цілісності щодо активного посередника більше не гарантується.

4.3. Чи можна прочитати або змінити дані

Активний зловмисник може надати власний сертифікат, встановити TLS-сеанс із клієнтом та окремий сеанс зі справжнім сервером. На своєму вузлі він розшифровує, читає або змінює дані та знову шифрує їх для іншої сторони. Коректні теги TLS у кожному сеансі не виявляють такої підміни кінцевої сторони.

4.4. Умови MITM атаки

Потрібна можливість спрямувати підключення клієнта до вузла зловмисника, наприклад через контроль точки Wi-Fi, шлюзу, проксі чи підміну DNS або ARP. Самого перебування у тій самій мережі недостатньо. Вимкнена перевірка дозволяє клієнту прийняти сертифікат посередника.

4.5. Правильне усунення помилки

Слід відновити перевірку й визначити причину збою: перевірити дату та час клієнта, Not Before/Not After, відповідність hostname полю SAN, підписи, проміжні сертифікати, trust store і, за потреби, статус відкликання. Сервер має надавати необхідні проміжні сертифікати; прострочений або невідповідний сертифікат потрібно замінити.

Для корпоративного CA його справжність перевіряють незалежним довіреним способом, після чого налаштовують довіру лише до необхідного CA відповідно до політики організації. Безумовне прийняття всіх сертифікатів не є виправленням [1, 4].

Висновки

Дослідження GitHub у браузері та онлайн-огляди Cloudflare показали застосування TLS 1.3 і AES-GCM. Для GitHub зафіксовано обмін ключами X25519, сертифікат із ключем EC та підписом ECDSA із SHA-256; браузер підтвердив чинність сертифіката і побудував ланцюжок довіри. Для Cloudflare наведено параметри з'єднання та сертифіката за результатами онлайн-перевірки.

Захист TLS залежить від узгодження параметрів, формування сеансових ключів та перевірки автентичності. CT, CRL, OCSP і HSTS доповнюють ці механізми. Heartbleed демонструє, що програмна помилка може розкрити дані з пам'яті навіть за стійких алгоритмів шифрування. Вимкнення перевірки сертифіката дозволяє активному посереднику отримати доступ до даних, тому необхідно усувати конкретну причину помилки.

Використані джерела

1. RFC 9846 — The Transport Layer Security Protocol Version 1.3
2. RFC 5869 — HMAC-based Extract-and-Expand Key Derivation Function
3. RFC 5246 — The Transport Layer Security Protocol Version 1.2
4. RFC 5280 — Internet X.509 Public Key Infrastructure Certificate and CRL Profile
5. RFC 8996 — Deprecating TLS 1.0 and TLS 1.1
6. RFC 7568 — Deprecating Secure Sockets Layer Version 3.0
7. RFC 6960 — X.509 Internet Public Key Infrastructure Online Certificate Status Protocol
8. RFC 9162 — Certificate Transparency Version 2.0
9. RFC 6797 — HTTP Strict Transport Security
10. OpenSSL Security Advisory — TLS heartbeat read overrun, 7 April 2014
11. digga — TLS certificate for github.com
12. digga — TLS certificate for cloudflare.com
13. The Heartbleed Bug
14. teardown.tools — SSL certificate for cloudflare.com